Mining Software Repositories

Chapter 5: Studying social aspects

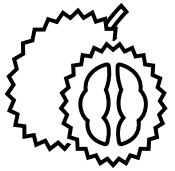
Prof. Dr. Steffen Herbold

Contents

- Community smells
- Diversity
- Conway’s law and socio-technical congruence

Smells in software engineering

- In software engineering, we use smell with a negative connotation
  - I.e., smell is always means smells bad and not smells good



- The usage of smell originates from the Refactoring book, where the term *code smell* was coined
- A code smell describes a pattern that indicates that the underlying code might have a deeper problem, e.g.
  - Feature envy: a component relies more on one other component then itself
    - Indicates that there may be a problem with the design due to a lack of cohesion
  - God class: a component that contains a huge amount of logic
    - Indicates that there is no clear separation of concerns
- Code smells are not bugs, but are associated with them (see Chapter 6)
- Code smells are a type of *technical debt*
  - Technical debt is an often-used metaphor in software engineering that the lack of the maintenance of the internal code quality leads to a debt in quality that will need to be paid at some point causing problems

Martin Fowler, Kent Beck: Refactoring: Improving the Design of Existing Code, Pearson Addison-Wesley

Smells from code smell to communities

- The notion of technical debt was adopted for the study of *social debt*
  - Social debt indicates that there are problems within a community that are not being addressed
  - Such problems may lead to frictions or other problems

- Based on the above, community smells can be seen as the counterpart to code smells for social debt
  - A code smell is a *pattern within source code* that indicates some deeper *problem within that code*
  - A community smell is a *pattern in the community structure* that indicates some deeper *problem within that community*
- The first set of community smells were identified by a qualitative study in which developers from a company were interviewed
  - The study was carried out based on open and axial coding of the interview responses
- This approach allowed deep insights into a single process and to generalize from there to define a set of community smells
- What is missing are broader perspectives on the relevance and prevalence of community smells

  → This is what MSR can provide!

Tamburri, D.A., Kruchten, P., Lago, P. *et al.* Social debt in software engineering: insights from industry. *J Internet Serv Appl* **6**, 10 (2015). https://doi.org/10.1186/s13174-015-0024-6

From code smells to community smells

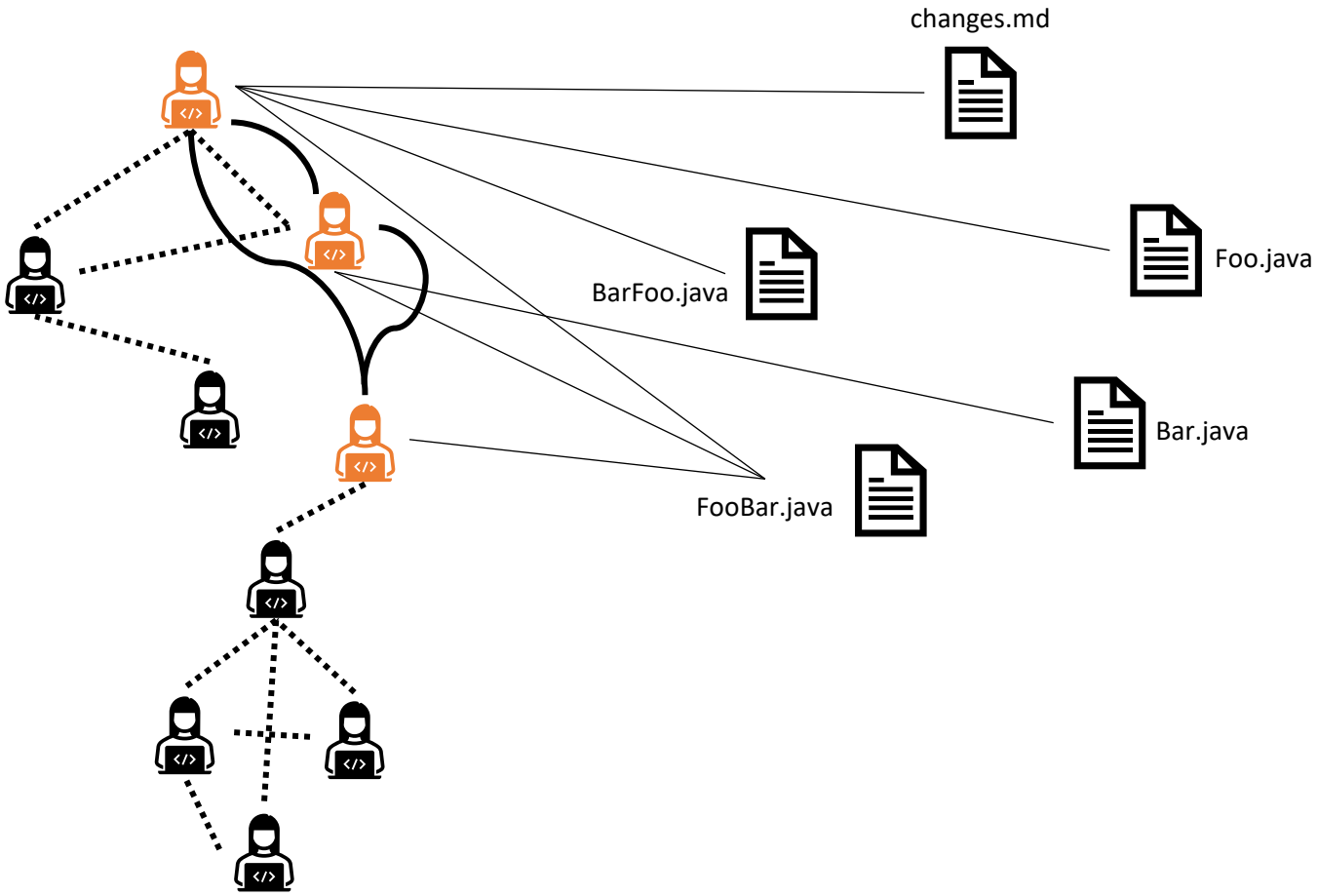
- Definition, detection, and monitoring of community smells can help to understand health of communities
  - Detection is often a precursor for corrective measures
  - Smells can inform activities that should increase the health of a community

- Since community smells are related to patterns in communication structure, developer social networks are a suitable tool to study them. We utilize two undirected graphs for this:
  - Communication graph $G_{comm} = (V_{comm}, E_{comm})$
    - The nodes are the developers taking part inf communication
    - An edge $e = (u, v) \in E_{comm}$ between two developers $u, v \in V_{comm}$ exists iff the developers communicated with each other in any medium (email, issue tracker, …)
  - Collaboration graph $G_{coll} = (V_{coll}, E_{coll})$
    - The nodes are the developers that modify source code elements
    - An edge $e = (u, v) \in E_{coll}$ between two developers $u, v \in V_{coll}$ exists, iff the developers worked on the same source code elements (i.e., files)

- We assume that $V_{coll} \subseteq V_{comm}$. Otherwise, we would have code contributorsthat are not communicating, which is an obvious problem anyways

D. A. Tamburri, F. Palomba and R. Kazman, "Exploring Community Smells in Open-Source: An Automated Approach," in *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630-652, 1 March 2021, doi: 10.1109/TSE.2019.2901490.

Example for community and collaboration graphs



Legend:

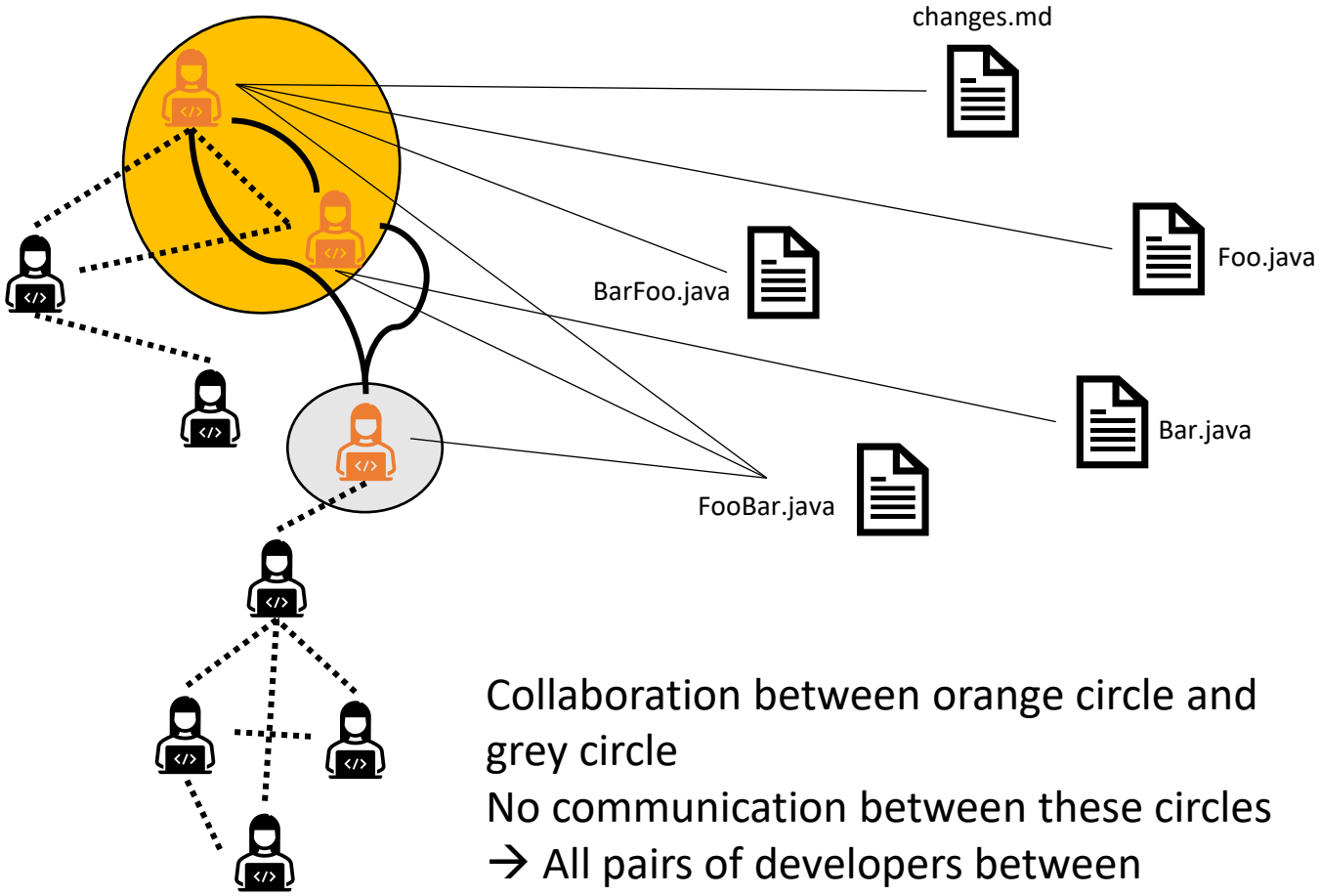
- Communicated with each other (edge in G_{comm})
- Worked on the same files (edge in G_{coll})
- Worked on file
- Node in collaboration and communication graph
- Node only in communication graph

Organizational silos

- A silo is an independent sub-community
- Silos often lead to duplication of effort
  - E.g., because the same problems are solved multiple times, lack of re-use of code between silos, or similar issues
- Silos can also have negative effects when this leads to egotistical behavior
  - E.g., prevent a required change to the architecture by others due to additional effort for own silo
- Using the communication graph, we can detect developers from different organizational silo as follows
  - $Silos = \{(u, v) : u, v \in V_{coll} : \text{no path from } u \text{ to } v \text{ exists in } G_{comm}\}$
- Through this, you can detect pairs of developers working on (not necessarily the same) code, that do not communicate with each other (not even indirectly via other developers)

D. A. Tamburri, F. Palomba and R. Kazman, "Exploring Community Smells in Open-Source: An Automated Approach," in *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630-652, 1 March 2021, doi: 10.1109/TSE.2019.2901490.

Organizational silos in our example



Collaboration between orange circle and grey circle
No communication between these circles
→ All pairs of developers between orange and grey are organizational silos

Legend:
- Communicated with each other (edge in G_{comm})
- Worked on the same files (edge in G_{coll})
- Worked on file
- Node in collaboration and communication graph
- Node only in communication graph

Lone wolfs

- If developers work on the same code, they should communicate with each other
- Otherwise, there is a big risk that a non-communicating developers makes decision about the code that are not in line with the architecture or design envisioned by the remainder of the team

- We call developers who act with such unchecked autonomy *lone wolfs*

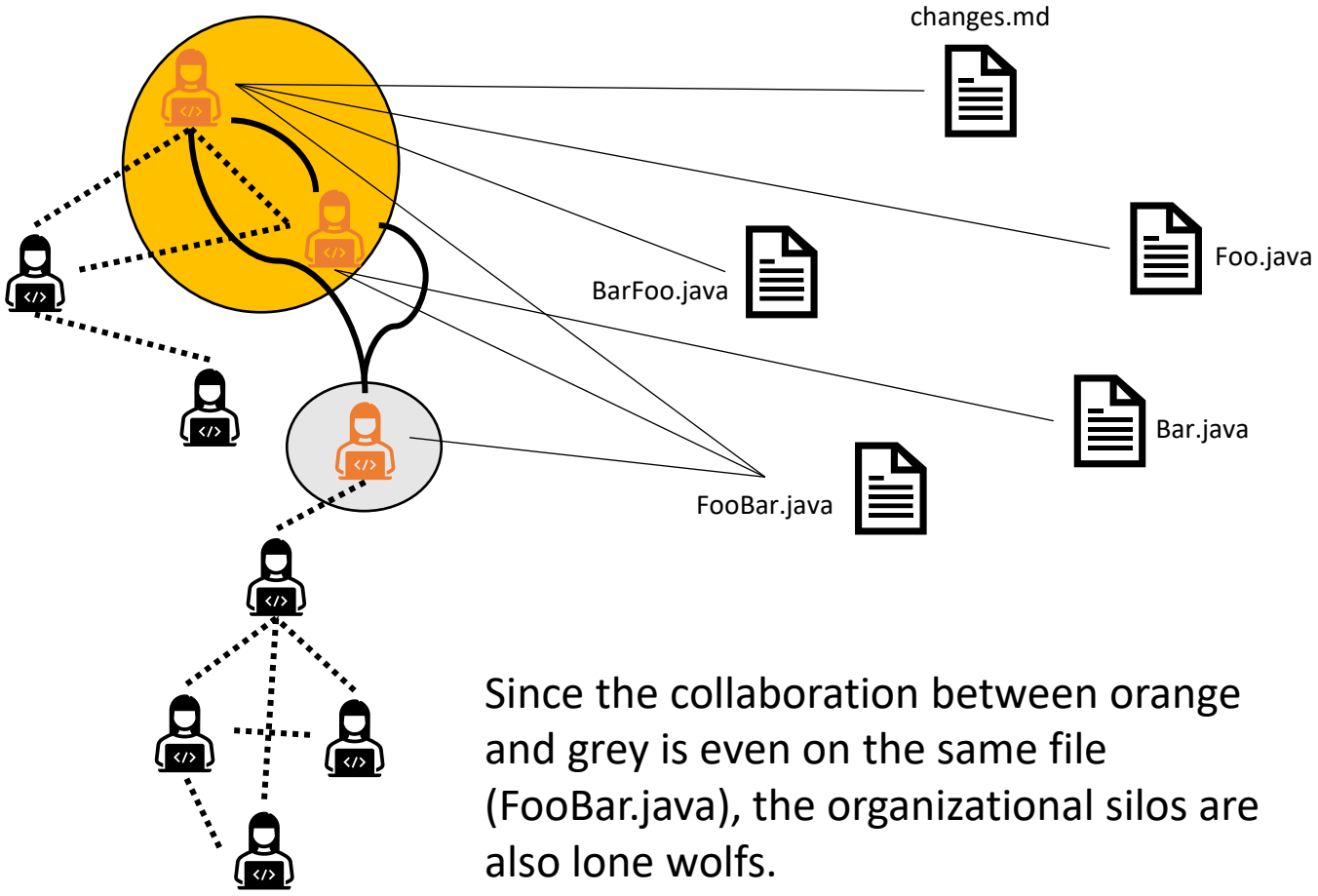
- Using the communication and collaboration graphs, we can describe the lone wolf as follows
  - $LoneWolfs = \{(u, v) \in E_{coll} : \text{no path from } u \text{ to } v \text{ exists in } G_{comm}\}$

- Through this, we can identify pairs of developers who work on the same code, but do not communicate with each other

- By definition, we have $LoneWolfs \subseteq Silos$

D. A. Tamburri, F. Palomba and R. Kazman, "Exploring Community Smells in Open-Source: An Automated Approach," in *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630-652, 1 March 2021, doi: 10.1109/TSE.2019.2901490.

Lone wolfs in our example



Since the collaboration between orange and grey is even on the same file (FooBar.java), the organizational silos are also lone wolfs.

Legend:

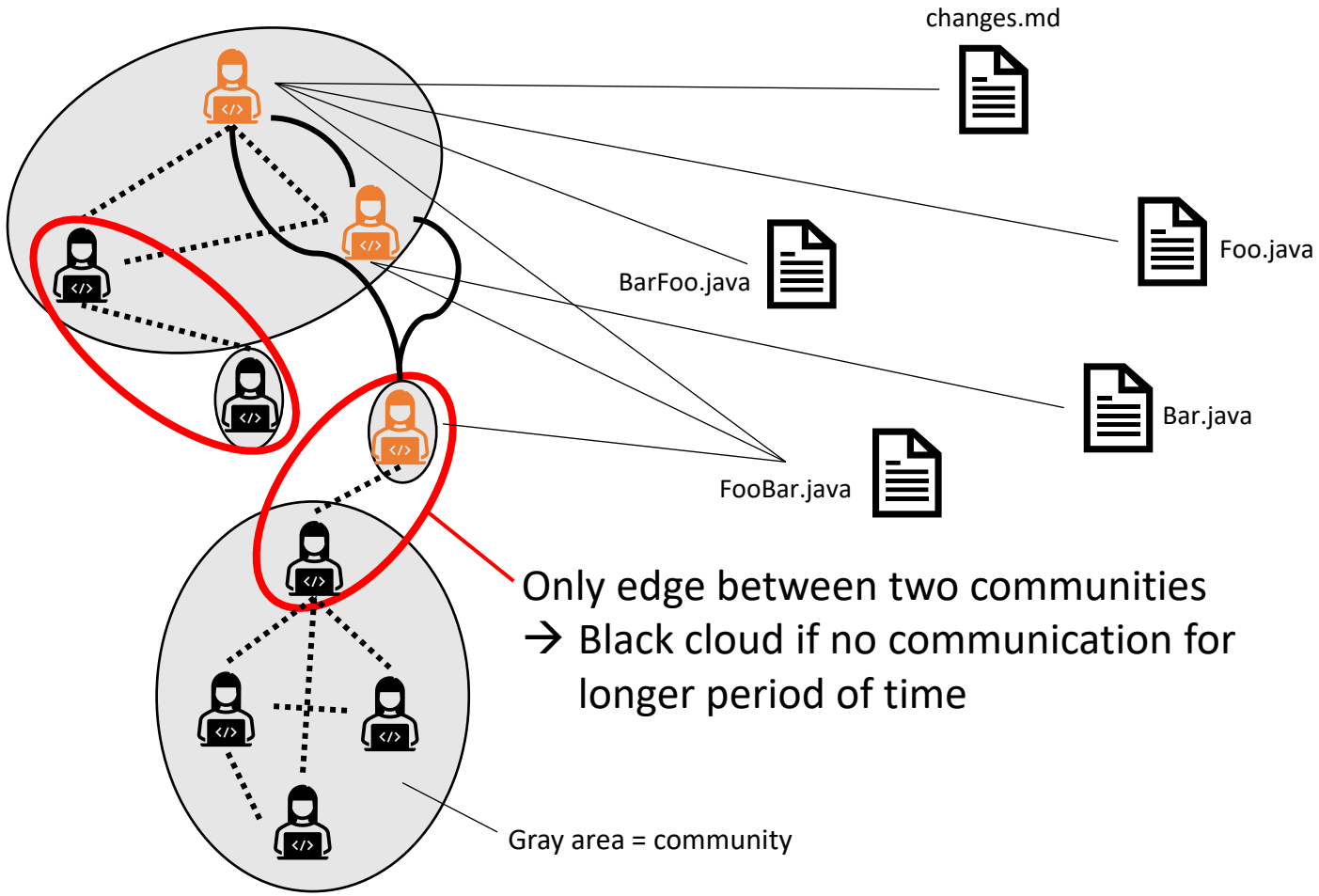
- Communicated with each other (edge in G_{comm})
- Worked on the same files (edge in G_{coll})
- Worked on file
- Node in collaboration and communication graph
- Node only in communication graph

Black clouds

- We say that there is a black cloud in the communication, if there is an inability to exchange knowledge and experiences between different (sub-)projects regularly
  - The name black cloud stems from the idea that a black cloud (e.g., due to a personal conflict) inhibits communication for a longer period of time
- In terms of graphs, we can capture the projects as communities
  - This is based on the idea that working together as a community defines a project
- We can identify developers that exchange information between communities
  - More specifically, for any pair of communities $p_i, p_j \in P = \{p_1, \dots, p_k\}, i \neq j$, with P being the communities of G_{comm} we can define the set of developers that communicate between these communities as
    - $bridge_{i,j} = \{(u, v) \in E_{comm} : u \in p_i \text{ and } v \in p_j\}$
  - If $\left|bridge_{i,j}\right| = 1$ there is only one pair of developers that communicates between the communities
- We say that there is a black cloud between communities p_i, p_j if $\left|bridge_{i,j}\right| = 1$ and these developers did not communicate for a longer time interval
  - The literature used 6 months as time interval

D. A. Tamburri, F. Palomba and R. Kazman, "Exploring Community Smells in Open-Source: An Automated Approach," in *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630-652, 1 March 2021, doi: 10.1109/TSE.2019.2901490.

Black clouds in our example



Legend:

- Communicated with each other (edge in G_{comm})
- Worked on the same files (edge in G_{coll})
- Worked on file
- Node in collaboration and communication graph
- Node only in communication graph

Bottlenecks (also called Radio Silence)

- Similar to a black cloud, a bottleneck indicates a problem in the communication between communities
- While black clouds look at the temporal component in information exchanges, bottlenecks consider only if there is a single-point of failure for the communication

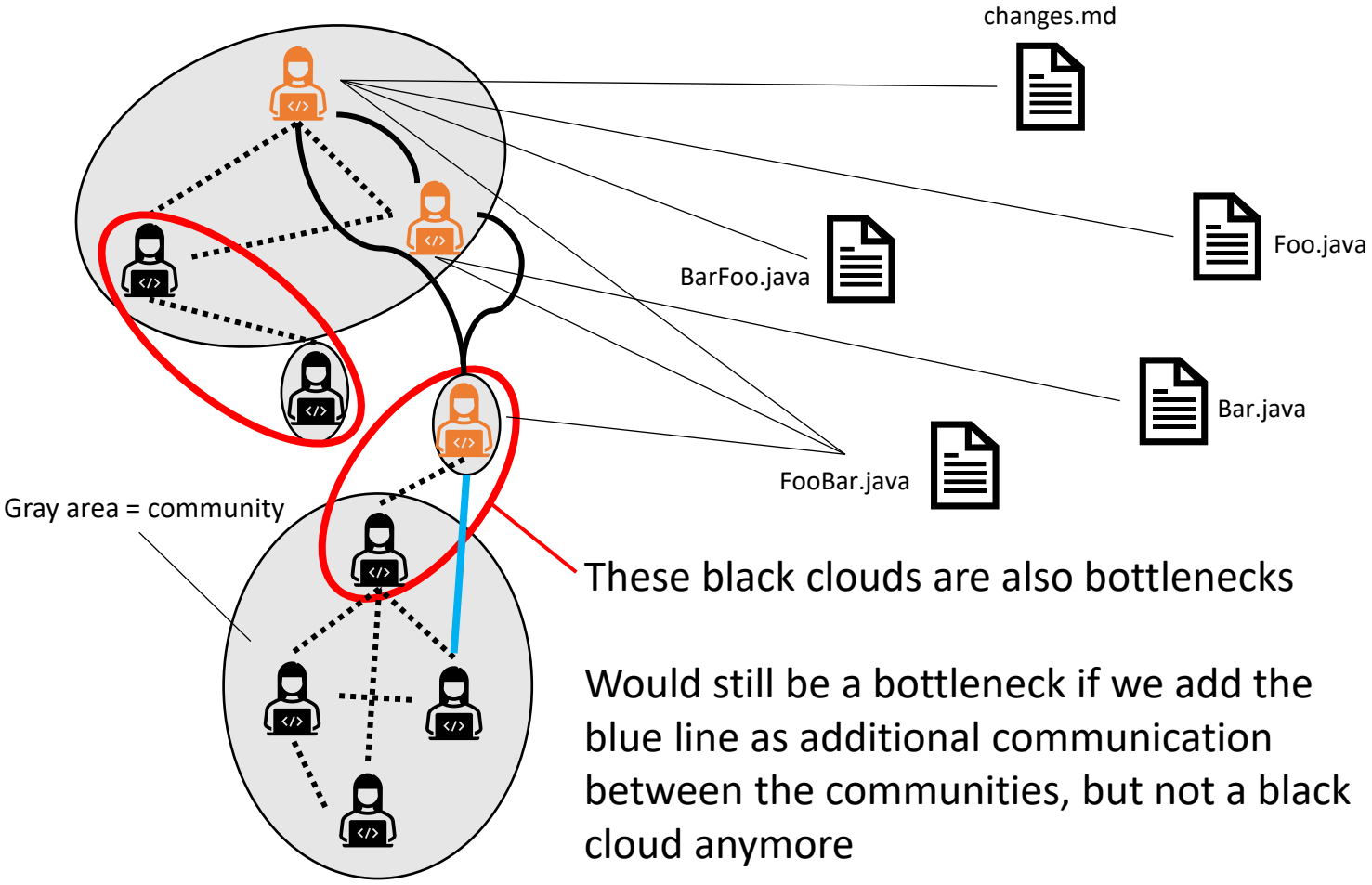
- This can be seen in a social network, if there is only a single developer in one community, that handles the communication with another community

- More specifically, for any pair of communities $p_i, p_j \in P = \{p_1, \dots, p_k\}, i \neq j$, with P being the communities of G_{comm} and the bridge between these communities $bridge_{i,j}$, we have a bottleneck between p_i, p_j if
  - $\left|\{v \in V_i : \exists (u, v) \in bridge_{i,j}\}\right| = 1$

- By definition, each black cloud is a bottleneck
  - If there is only one edge between communities, there can also be only developer handling the communication

D. A. Tamburri, F. Palomba and R. Kazman, "Exploring Community Smells in Open-Source: An Automated Approach," in *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 630-652, 1 March 2021, doi: 10.1109/TSE.2019.2901490.

Bottlenecks in our example



These black clouds are also bottlenecks

Would still be a bottleneck if we add the blue line as additional communication between the communities, but not a black cloud anymore

Legend:

- Communicated with each other (edge in G_{comm})
- Worked on the same files (edge in G_{coll})
- Worked on file
- Node in collaboration and communication graph
- Node only in communication graph

Diversity

Impact of diversity on software engineering

- Diversity is a broad concept. Among others, there are the following aspects:
  - Gender diversity
  - Cultural diversity
  - Sexual orientation

- In terms of software development, diversity is a social aspect
  - The above aspects are all not related to competency
  - The above aspects may all affect communication and collaboration

- As an example, we consider a study of the relation between the number of women on a team and community smells

Gender diversity vs community smells

- Researchers considered the relationship between gender diversity and community smells

- They had the following hypothesis based on prior management literature on gender diversity
  - “The presence of women within the team improves communication, co-operation, and collaboration, thus reducing the number of community smells”

- This was studied using a statistical model
- Such a model typically relies on three types of variables:
  - Dependent variables as the factor on which an impact is considered based on some hypothesis. In this case the community smells
  - Independent variables as factors that should (or should not) have an influence on the dependent variables based on some hypothesis. In this case this is the gender.
  - Confounding variables as additional factors that could also explain changes in the dependent variable. In this case such factors are, e.g., related to the project size and project age.

Catolino, Gemma, et al. "Gender diversity and women in software teams: How do they affect community smells?." 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS). IEEE, 2019.

Modeling the relationship between gender and community smells

Dependent variables:
- Number of organizational silos
- Number of lone wolfs
- Number of black clouds
- Number of bottlenecks

Independent variables:
- Number of women in a team
- Blau-Index ($1 - p_{male}^2 - p_{female}^2$)

Confounding/control variables:
- Number of committers
- Number of commits
- Team size
- Turnover of developers
- Project age
- Tenure diversity
  - differences in experience within project
- Tenure median
- Ratio of collaborations and communications ($\frac{|E_{coll}|}{|E_{comm}|}$)
- Truck factor
  - How many developers can be removed before the project get disconnected
- Centrality

Questions that the statistical model should answer:
- Which of the independent and confounding variables explain the value of the dependent variables?
- How strong is the influence of the different variables with an influence?

Model used here:
- Linear model, i.e., linear regression of the dependent variable based on the other variables
- Statistical test (usually some variant of ANOVA) to determine if coefficients are significant
  - Significant means that with a certain probability (usually more than 95%) the coefficient is not zero
  - Not zero means we have a measurable effect
- Values of significant coefficient explain the influence of the different variables

Catolino, Gemma, et al. "Gender diversity and women in software teams: How do they affect community smells?." 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS). IEEE, 2019.

Results for bottlenecks

Table III: Results achieved by the three models built for Radio Silence. If the values for a certain variable are not presented, it means that the *vif* function removed it from the model.

Factor	All Variables			Conf. Variables			Random		
	Estimate	S.E	Sig.	Estimate	S.E	Sig.	Estimate	S.E	Sig.
(Intercept)	-2.260	0.635		-2.743	0.624		2.335	0.128	
log(totalCommitters)	-0.049	0.085		-0.072	0.080				
log(totalcommits)	0.027	0.060		0.022	0.061				
projectAge	-0.025	0.015	.	-0.018	0.015				
turnover	10.048	0.455	***	10.119	0.458	***			
blauGender	0.194	0.804							
tenureMedian	0.044	0.035		0.034	0.035				
tenureDiversity	0.010	0.025		0.007	0.025				
stCongruence	-0.308	0.186	.	-0.265	0.187				
centrality	-0.038	0.146		-0.046	0.146				
log(teamSize)				0.195	0.068	**			
truckFactor	-0.014	0.046		-0.009	0.046				
numberOfWomen	0.038	0.012	**						
****p < 0.001 - **p < 0.01 - *p < 0.05 .p < 0.1*									

Developer turnover is the main driver behind this smell

More women reduce the likelihood of this smell

Results for lone wolfs

Table V: Results achieved by the three models built for Lone Wolf. If the values for a certain variable are not presented, it means that the *vif* function removed it from the model.

Factor	All Variables			Conf. Variables			Random		
	Estimate	S.E	Sig.	Estimate	S.E	Sig.	Estimate	S.E	Sig.
(Intercept)	-8.462	4.112		-8.679	4.123		4.456	0.180	
log(totalCommitters)	0.267	0.289		0.414	0.262				
log(totalcommits)	0.121	0.219		0.086	0.217				
projectAge	0.089	0.051	.	0.068	0.049				
turnover	0.253	1.002		-0.193	0.929				
blauGender	-3.376	2.939							
tenureMedian	-0.031	0.121		-0.0313	0.122				
tenureDiversity	0.078	0.087		0.069	0.086				
stCongruence	-8.298	1.511	***	-8.465	1.510	***			
centrality	0.084	0.269		0.135	0.272				
log(teamSize)	4.315	0.892	***	4.250	0.893	***			
truckFactor	-0.168	0.159		-0.177	0.159				
numberOfWomen	0.013	0.038	.						
****p < 0.001 - **p < 0.01 - *p < 0.05 .p < 0.1*									

Explained by team size and ratio of collaborations/communications

No relation to number of women or Blau-index

Catolino, Gemma, et al. "Gender diversity and women in software teams: How do they affect community smells?." 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS). IEEE, 2019.

Conway‘s law and socio-technical congruence

Organization structure and design

> “Organizations which design systems are constrained to produce designs which are **copies of the communication structures** of these organizations”
> – Conway, 1968

This means that our system design would mimic team structures, which we also refer to as *socio-technical congruence*

→ That is something we can model with a social-technical network!

Conway, Melvin E. "How do committees invent." *Datamation* 14.4 (1968): 28-31.

Conway‘s law and communities

- Can be studied by looking at communities
  - We already used communities for smells earlier based on the idea that these are developers that are working closely together
- Based on Conway‘s law, there should be a relationship between the communities and the architecture
- For example, research found that there are two communities within the Apache web server development
  - Code-oriented developers
  - Documentation-oriented developers
- The social structure represents this work on different project components

Del Rosso, Christian. "Comprehend and analyze knowledge networks to improve software evolution." *Journal of software maintenance and evolution: research and practice* 21.3 (2009): 189-215.

Coordination requirements

- Not every communication is relevant for a project
  - For example, an informal exchange in a chat (“is it still raining outside?”, “want to go to lunch?”, …) is not related to relevant communication for the coordination within a project
    - One could argue that project aspects might be discussed at informal meetings, but this would be a different research topic
- A coordination requirement is related to communication that needs to happen within a project
  - For example, when two developers work on the same component or feature, they need to coordinate
  - Similarly, when one developers reports a bug to another developer, they need to coordinate
- Consequently, communication in issue tracking systems is suitable to model coordination requirements
- Each issue represents a task
  - Feature addition, bug fix, …

Marcelo Cataldo, James D. Herbsleb, and Kathleen M. Carley. 2008. Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity. In Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM '08). Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/1414004.1414008

Modeling architectures

- The nodes of an architecture network are usually the files within a project
  - While this is usually more fine-grained than actual architecture, we should observe the same boundaries as within the actual architecture
  - Could also be based on packages/folders as a more coarse-grained structure instead

- Static structure to model architecture
  - The edges in the architecture are defined by usage relationships on the code level
  - In practice, we usually rely on import statements: one file uses another, if it is imported
    - Since imports can be too coarse (e.g., import java.util.*) or unused, a cleaner but more difficult to obtain variant would be to use the call graph

- Co-change to model architecture
  - Alternatively, we can also model architectures through co-change
  - In this case, the we assume that components that are changed together should belong to the same architectural components

Marcelo Cataldo, James D. Herbsleb, and Kathleen M. Carley. 2008. Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity. In Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM '08). Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/1414004.1414008

Modelling coordination requirements

- Task assignments
  - Assignment of issues to developers is usually handled using the metadata of an issue and can be identified there
  - $Task^{assign} = \left(t_{ij}^{A}\right) \in \{0,1\}^{n_{devs} \times n_{tasks}}$ such that $t_{ij}^{A} = 1$ is developer i worked on task j
- Task dependencies
  - Tasks depend on software artifacts
    - For example, when a feature should be added, the task depends on all files that need to be touched to add that feature
  - Dependencies can also be indirect, i.e., the changed components depend on architecturally related components
    - For example, when you work in a package, this is usually related to other code in the same package
  - Tasks can be interdependent
    - If two features are added to the same files, these tasks depend on each other
    - There is also the indirect dependency when architecturally related files are changed
  - $Task^{depends} = \left(t_{ij}^{D}\right) \in \mathbb{N}_{0}^{n_{tasks} \times n_{tasks}}$ such that $t_{ij}^{D} = k$ means that tasks i, j have an overlap of k files that are directly or indirectly depend
- Coordination requirements
  - We can say that developers need to coordinate, if they are assigned to dependent tasks
  - We can further argue that if there are more dependencies in terms of overlapping files, the requirement to coordinate is stronger
  - $Coord^{req} = Task^{assign} Task^{depends} \left(Task^{assign}\right)^{T} \in \mathbb{N}_{0}^{n_{devs} \times n_{devs}}$

Marcelo Cataldo, James D. Herbsleb, and Kathleen M. Carley. 2008. Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity. In Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM '08). Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/1414004.1414008

Coordination activities

- Coordination activities model when developers actually communicate to coordinate

- This is a typical social network was we know it from before
  - Nodes are developers
  - Edges are communication between these developers, e.g., within an issue

- We represent this as an adjacency matrix
  - $Coord^{actual} = \left(c_{ij}^{A}\right) \in \{0,1\}^{n_{devs} \times n_{devs}}$ such that $c_{ij}^{A} = 1$ if there was coordination between developers i, j

Marcelo Cataldo, James D. Herbsleb, and Kathleen M. Carley. 2008. Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity. In Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM '08). Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/1414004.1414008

Computing congruence

- We know have modeled who should coordinate with each other ($Coord^{req}$) and who does coordinate with each other ($Coord^{actual}$)
- We can measure the congruence between both by looking at the ratio of the required coordination that took place
  - $Congruence\left(Coord^{req}, Coord^{actual}\right) = \frac{\#CoordHappend\left(Coord^{req}, Coord^{actual}\right)}{\#CoordRequired\left(Coord^{req}\right)}$
  - $\#CoordHappend\left(Coord^{req}, Coord^{actual}\right) = \left|\{(i,j) \in \{1, \dots, n_{devs}\}^2 : c_{ij}^R > 0 \; and \; c_{ij}^A > 0\}\right|$
  - $\#CoordRequired\left(Coord^{req}\right) = \left|\{(i,j) \in \{1, \dots, n_{devs}\}^2 : c_{ij}^R > 0\}\right|$

- Social-technical congruence is closely related Conway‘s law
  - The coordination requirements are derived from the architectural dependencies
  - Values close to one mean that the project communication matches the requirements
  - The further away a value is from one, the greater the lack of internal communication which means that the architecture does not follow Conway‘s law

Marcelo Cataldo, James D. Herbsleb, and Kathleen M. Carley. 2008. Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity. In Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM '08). Association for Computing Machinery, New York, NY, USA, 2–11. https://doi.org/10.1145/1414004.1414008

Questions?

